

Typing Tutor

How to compile and run

```
make
```

This automatically compiles all `.java` files and launches the application.

Other make targets

```
make compile # Compile only
make run # Run without compiling
make clean # Remove all compiled .class files
```

How to use

1. A pangram prompt is displayed at the top in blue.
2. Click inside the **Your Input** text area and start typing.
3. Each key you press is highlighted on the virtual keyboard.
4. Your accuracy is updated in real time after every keystroke.
5. The **Difficulty Report** panel shows keys you have missed, sorted by miss count.
6. Press **TAB + ENTER** or click **Restart** to get a new prompt and reset all stats.

Keyboard Shortcuts

Shortcut	Action
TAB + ENTER	Restart with a new prompt

Class Overview

Class	File	Responsibility
TypingTutor	TypingTutor.java	Main window, UI layout, game state management
VirtualKB	VirtualKB.java	Virtual keyboard rendering and key highlighting
TypingKeyListener	TypingKeyListener.java	Keyboard event handling and input validation

TypingTutor (extends JFrame)

The entry point and central controller of the application. Builds the UI, manages game state, and coordinates between the virtual keyboard and key listener.

Fields

Field	Type	Description
PANGRAMS	String[]	Static list of pangrams used as typing prompts
TARGET	String	The currently active prompt the user must type
inputTextArea	JTextArea	Editable area where the user types
reportTextArea	JTextArea	Read-only panel showing the difficulty report
promptLabel	JLabel	Displays the current target prompt in blue
statsLabel	JLabel	Displays live accuracy, correct count, and total count
resetButton	JButton	Clicking this calls <code>restart()</code>
virtualKB	VirtualKB	The on-screen keyboard component
keyListener	TypingKeyListener	The keyboard event listener attached to <code>inputTextArea</code>
missedKeysMap	Map<Character, Integer>	Tracks how many times each key was missed
isShifted	boolean	Whether Shift is currently held
isCaps	boolean	Whether Caps Lock is currently active
correctKeys	int	Number of correctly typed keystrokes so far
totalKeys	int	Total number of keystrokes attempted so far

Methods

Method	Description
<code>TypingTutor()</code>	Constructor. Builds the full UI layout, initializes all components, attaches the key listener, and detects the initial Caps Lock state.
<code>checkAcc(char typedChar)</code>	Called on every keystroke. Compares the typed character against the expected character in <code>TARGET</code> at the current cursor position. Increments <code>correctKeys</code> if correct, otherwise records the miss in <code>missedKeysMap</code> and calls <code>updateDiffReport()</code> . Updates <code>statsLabel</code> with the latest accuracy.

Method	Description
<code>setShiftState(boolean shiftActive)</code>	Updates <code>isShifted</code> and tells <code>VirtualKB</code> to re-render key labels accordingly.
<code>toggleCapsState()</code>	Flips <code>isCaps</code> and tells <code>VirtualKB</code> to re-render key labels.
<code>restart()</code>	Resets all state: clears pressed keys, re-detects Caps Lock, picks a new prompt via <code>selectRandom()</code> , clears the input area, resets counters, and returns focus to the input area.
<code>updateDiffReport()</code>	Rebuilds the text in <code>reportTextArea</code> from <code>missedKeysMap</code> , sorted by miss count descending. Shows a default message if no errors have occurred.
<code>selectRandom()</code>	Picks a random pangram from <code>PANGRAMS</code> , ensuring it differs from the current <code>TARGET</code> when more than one option is available.
<code>main(String[] args)</code>	Entry point. Launches <code>TypingTutor</code> on the Swing Event Dispatch Thread via <code>SwingUtilities.invokeLater</code> .

VirtualKB (extends JPanel)

Renders a five-row QWERTY keyboard as a grid of `JBUTTON` components. Handles key highlighting and label toggling for Shift and Caps Lock states.

Fields

Field	Type	Description
<code>keyButtonMap</code>	<code>Map<Integer, JButton></code>	Maps each key code to its corresponding button on screen
<code>shiftMap</code>	<code>Map<Integer, String[]></code>	Maps each key code to a two-element array: <code>[defaultLabel, shiftedLabel]</code>
<code>defaultButtonColor</code>	<code>Color</code>	Stores the original button background color for restoration after highlighting

Methods

Method	Description
<code>VirtualKB()</code>	Constructor. Sets up the panel layout and calls

Method	Description
	<code>buildVirtualKB()</code> .
<code>buildVirtualKB()</code>	Defines all five keyboard rows as data arrays and calls <code>storeShiftData()</code> and <code>createKbRow()</code> for each.
<code>storeShiftData(Object[][] rowData)</code>	Populates <code>shiftMap</code> with the default and shifted label for each key in a row.
<code>createKbRow(Object[][] keyData)</code>	Creates a <code>JPanel</code> for one keyboard row, instantiates a <code>JButton</code> for each key, stores it in <code>keyButtonMap</code> , and returns the panel.
<code>highlightKey(int keyCode, Color color, boolean opaque)</code>	Changes the background color of the button matching <code>keyCode</code> to indicate it is being pressed.
<code>restoreKeyColor(int keyCode)</code>	Resets the button matching <code>keyCode</code> back to <code>defaultButtonColor</code> when the key is released.
<code>toggleLabels(boolean shiftActive, boolean capsActive)</code>	Iterates all buttons and updates their labels. Letter keys follow XOR logic (<code>shiftActive ^ capsActive</code>) for correct Caps Lock and Shift interaction; symbol keys simply switch between default and shifted labels.

TypingKeyListener (implements KeyListener)

Handles all keyboard events from the input text area. Coordinates key highlighting on the virtual keyboard, accuracy checking, Shift/Caps state changes, and the restart shortcut.

Fields

Field	Type	Description
<code>app</code>	<code>TypingTutor</code>	Reference to the main application for calling state-change methods
<code>kb</code>	<code>VirtualKB</code>	Reference to the virtual keyboard for highlighting
<code>highlightColor</code>	<code>Color</code>	The color applied to a key button when pressed
<code>pressed</code>	<code>Set<Integer></code>	Tracks all key codes currently held down, enabling multi-key detection
<code>restarted</code>	<code>boolean</code>	Flag that suppresses the spurious <code>keyTyped</code> event that fires immediately after a TAB

Field	Type	Description
		+ ENTER restart

Methods

Method	Description
<code>TypingKeyListener(TypingTutor app, VirtualKB kb, Color highlightColor)</code>	Constructor. Stores references to the main app, virtual keyboard, and highlight color.
<code>keyTyped(KeyEvent e)</code>	Fires when a printable character is generated. Consumes the event silently if <code>restarted</code> is true or if TAB/ENTER is held. Otherwise calls <code>app.checkAcc()</code> for valid non-backspace characters.
<code>keyPressed(KeyEvent e)</code>	Fires when any key is pressed. Adds the key code to <code>pressed</code> , triggers <code>app.restart()</code> if both TAB and ENTER are held simultaneously, updates Shift/Caps state, and highlights the key on the virtual keyboard.
<code>keyReleased(KeyEvent e)</code>	Fires when a key is released. Removes the key code from <code>pressed</code> , clears the <code>restarted</code> flag if no keys remain held, updates Shift state, and restores the key's original color on the virtual keyboard.
<code>clearPressedKeys()</code>	Empties the <code>pressed</code> set. Called by <code>app.restart()</code> to prevent ghost key states after a reset.